JAVASCRIPT COMPLETE ROADMAP

Numbered subtopics edition — 1.1, 1.2, 1.3…

6 PHASES Beginner to professional	**HIERARCHICAL** Topics and subtopics numbered
NO BULLETS Clean numbered structure	**DARK EDITION** Black canvas, orange accents

Prepared from the supplied JavaScript roadmap. Topic sequence is retained; subtopics are presented as numbered entries instead of bullets.

Author: OpenAI | Edition: October 2026

CONTENTS

Each phase uses hierarchical labels so individual topics are easy to reference.

01	Introduction & prerequisites
02	Phase 1 — Foundations
03	Phase 2 — Built-in objects & utilities
04	Phase 3 — Modern & advanced JavaScript
05	Phase 4 — Web development & asynchronous programming
06	Phase 5 — Interview depth & professional JavaScript
07	Phase 6 — Capstone projects
08	Timeline & milestones
09	Resources
10	Glossary
11	Appendix

Progress rule: move on when you can explain a concept and apply it in a small working example.

01 · Introduction & prerequisites

The roadmap moves from core syntax to language internals, browser APIs, professional practices, and larger projects.

1.1 How to use this roadmap

1.1.1 Study one numbered topic group at a time and write examples.

1.1.2 Build or extend a small project after each phase.

1.1.3 Revisit earlier topics when later concepts reveal gaps.

1.1.4 Treat less-used concepts as exploration topics after fundamentals.

1.2 Practical prerequisites

1.2.1 A computer with a modern browser and developer tools.

1.2.2 A code editor and a folder for JavaScript practice files.

1.2.3 Basic comfort with files and folders; command-line familiarity is helpful.

1.2.4 No prior JavaScript knowledge is required for Phase 1.

The timeline is a suggested pace, not a duration stated in the original source.

02 · Phase 1 — Foundations

Goal: use core syntax, data structures, functions, errors, and the DOM in small programs.

2.1 Basic syntax & structure

2.1.1 Comments
2.1.2 Variables: var, let, const
2.1.3 Data types: String, Number, Boolean, Null, Undefined, Object, Array

2.2 Operators

2.2.1 Arithmetic
2.2.2 Comparison
2.2.3 Logical
2.2.4 Assignment

2.3 Control structures

2.3.1 if / else
2.3.2 switch

2.4 Loops

2.4.1 for
2.4.2 while
2.4.3 do-while
2.4.4 for-in
2.4.5 for-of

2.5 Functions

2.5.1 Function declaration
2.5.2 Function expression
2.5.3 Arrow (fat arrow) function
2.5.4 Anonymous function
2.5.5 IIFE (Immediately Invoked Function Expression)
2.5.6 Four ways to write a function
2.5.7 Default parameters
2.5.8 Return statement
2.5.9 Parameters and arguments

2.6 Arrays

2.6.1 Creating and accessing arrays
2.6.2 Array.isArray()
2.6.3 Array.to() — listed in the source; verify intended meaning
2.6.4 Array.from()
2.6.5 push() and pop()
2.6.6 shift() and unshift()
2.6.7 slice() and splice()
2.6.8 join()
2.6.9 forEach()
2.6.10 map()
2.6.11 filter()
2.6.12 reduce() and other methods

2.7 Objects

2.7.1 Creating objects
2.7.2 Accessing properties and methods
2.7.3 Nested objects

2.7.4 Object.keys()
2.7.5 Object.values()
2.7.6 Object.entries()
2.7.7 Object.assign()
2.7.8 Object.freeze()
2.7.9 Object.seal()

2.8 Basic error handling

2.8.1 try, catch, finally
2.8.2 Custom errors

2.9 Basic DOM manipulation

2.9.1 Selecting and changing page elements
2.9.2 Using JavaScript to respond to page content

Phase 1 milestone: write programs with conditions, loops, functions, arrays, and objects, then create a page that responds to DOM changes.

03 · Phase 2 — Built-in objects & utilities

Goal: use built-in objects and global functions for numeric, date, and data-conversion tasks.

3.1 Math object

3.1.1 Math.round()
3.1.2 Math.floor()
3.1.3 Math.ceil()
3.1.4 Math.random()
3.1.5 Math.max()
3.1.6 Math.min()
3.1.7 Math.pow()
3.1.8 Math.sqrt()
3.1.9 Math.abs()
3.1.10 Other Math methods

3.2 Date object

3.2.1 new Date()
3.2.2 Date.now()
3.2.3 getFullYear()
3.2.4 getMonth()
3.2.5 getDate()
3.2.6 getDay()
3.2.7 setFullYear()
3.2.8 setHours()
3.2.9 Other Date methods

3.3 Date and time logic

3.3.1 Date formatting
3.3.2 Calculating time differences

3.4 JSON

3.4.1 JSON.stringify() — object to JSON string
3.4.2 JSON.parse() — JSON string to object

3.5 Global functions

3.5.1 parseInt()
3.5.2 parseFloat()
3.5.3 isNaN()
3.5.4 isFinite()
3.5.5 encodeURIComponent()
3.5.6 decodeURIComponent()
3.5.7 Other global functions

Phase 2 milestone: create a date/time utility and practice converting JavaScript objects to JSON and back.

04 · Phase 3 — Modern & advanced JavaScript

Goal: understand JavaScript behavior and use modern language features confidently.

4.1 Basic understanding

4.1.1 let vs const vs var
4.1.2 Pass by value vs pass by reference
4.1.3 Truthy vs falsy
4.1.4 Different types of export and import

4.2 Modern JavaScript

4.2.1 ECMAScript
4.2.2 Symbols
4.2.3 BigInt
4.2.4 Sets
4.2.5 Maps
4.2.6 Iterators and generators
4.2.7 Proxy and Reflect
4.2.8 Spread and rest syntax
4.2.9 Destructuring
4.2.10 Nullish coalescing
4.2.11 Optional chaining
4.2.12 Modules
4.2.13 WeakMap and WeakSet
4.2.14 Dynamic imports
4.2.15 Top-level await

4.3 Advanced JavaScript

4.3.1 Function hoisting
4.3.2 Closures
4.3.3 Type equality and coercion
4.3.4 Short-circuit evaluation
4.3.5 Callback functions
4.3.6 Constructor functions
4.3.7 Objects and prototypes
4.3.8 Classes and inheritance
4.3.9 Functional programming
4.3.10 Object-oriented programming
4.3.11 Autoboxing and primitive wrapper objects

Phase 3 milestone: refactor a project using modules and modern syntax; explain closures, prototypes, coercion, and value/reference behavior.

05 · Phase 4 — Web development & asynchronous programming

Goal: build browser experiences and understand how asynchronous work is scheduled.

5.1 Website development

5.1.1 Node.js vs browser
5.1.2 Global object
5.1.3 DOM manipulation
5.1.4 Event propagation and handling
5.1.5 Form handling
5.1.6 localStorage
5.1.7 sessionStorage
5.1.8 IndexedDB
5.1.9 Service Workers
5.1.10 Web Workers
5.1.11 WebSockets
5.1.12 WebGL API and Canvas API
5.1.13 Other Web APIs
5.1.14 Accessibility (a11y) basics

5.2 Asynchronous programming

5.2.1 Synchronous vs asynchronous execution
5.2.2 JavaScript engine and JavaScript environment
5.2.3 Timers
5.2.4 Call stack
5.2.5 Event loop
5.2.6 Task queue and microtask queue
5.2.7 Callback hell
5.2.8 Promises
5.2.9 Async / await
5.2.10 AbortController
5.2.11 Concurrency vs parallelism

Phase 4 milestone: build a browser app that handles forms, persists data, and fetches data asynchronously with loading and error states.

06 · Phase 5 — Interview depth & professional JavaScript

Goal: explain runtime behavior, debug complex issues, test code, and ship applications.

6.1 Interview topics

6.1.1 Execution context
6.1.2 this keyword
6.1.3 call(), apply(), bind()
6.1.4 Static vs dynamic typing
6.1.5 TypeScript overview: type safety for JavaScript
6.1.6 JIT compilation
6.1.7 WebAssembly (WASM) integration
6.1.8 Memory management
6.1.9 Event propagation: bubbling, capturing, and event delegation
6.1.10 Macrotasks vs microtasks in detail
6.1.11 Memory leaks and garbage collection
6.1.12 Deep vs shallow copy

6.2 Professional JavaScript

6.2.1 JavaScript design patterns
6.2.2 Advanced debugging techniques and tools
6.2.3 Best practices and code quality
6.2.4 Unit testing
6.2.5 Deploying JavaScript applications
6.2.6 Git and GitHub for version control

Phase 5 milestone: explain runtime behavior, debug an issue, write tests, use Git history, and deploy a small application.

07 · Phase 6 — Capstone projects

The source recommends building three or four substantial projects. The numbered items below preserve its four project ideas.

7.1 Portfolio website

7.1.1 Use semantic HTML and accessible navigation.

7.1.2 Add responsive styling and useful interactions.

7.1.3 Publish the site and keep source control in GitHub.

7.2 To-do app with localStorage or IndexedDB

7.2.1 Create, complete, edit, and delete tasks.

7.2.2 Persist tasks after refresh.

7.2.3 Handle empty states and invalid input.

7.3 Real-time chat app (WebSockets + Service Workers)

7.3.1 Build a chat interface and connection states.

7.3.2 Handle messages, reconnects, errors, and user feedback.

7.3.3 Explore Service Workers where appropriate.

7.4 E-commerce mini app (cart, auth, payment simulation)

7.4.1 Display products and manage a shopping cart.

7.4.2 Model a basic authentication flow and checkout state.

7.4.3 Use a simulated payment flow; do not collect real payment details in a demo.

Project quality checklist: README, clear folder structure, meaningful commits, tests where appropriate, and a deployed demo when feasible.

08 · Timeline & milestones

Suggested pacing only; adjust it to your schedule and confidence applying the concepts.

Stage	Suggested pace	Evidence of progress
Phase 1	3–5 weeks	Syntax exercises and a DOM mini project
Phase 2	1–2 weeks	Date/time utility and JSON practice
Phase 3	3–5 weeks	Modular refactor; explain advanced behavior
Phase 4	3–5 weeks	Async browser app with persistent data
Phase 5	3–5 weeks	Tested, version-controlled deployment
Phase 6	4–8+ weeks	Three or four portfolio-ready projects

Estimated total: approximately 17–30+ weeks at a steady pace. This estimate is added for planning and was not specified by the source.

8.1 Review questions

8.1.1 Can I build a small example without copying a tutorial line by line?
8.1.2 Can I explain why the code behaves this way?
8.1.3 Can I handle an error or unexpected input?
8.1.4 Can I describe what I would improve next?

09 · Resources

The original roadmap did not include named books, courses, websites, or video channels.

9.1 What to look for

9.1.1 A current JavaScript reference covering syntax, built-ins, modules, and browser APIs.
9.1.2 Hands-on exercises with explanations and edge cases.
9.1.3 Documentation for DOM, storage, workers, WebSockets, and accessibility.
9.1.4 Guides for testing, debugging, Git, and GitHub.

9.2 How to evaluate a resource

9.2.1 Check whether examples distinguish browser behavior from Node.js behavior.
9.2.2 Prefer explanations that show expected results and common mistakes.
9.2.3 Use official documentation for API details and tutorials for guided practice.

10 · Glossary

Short explanatory definitions for terms appearing in the roadmap.

10.1 API	A defined interface that lets code use a service or platform capability.
10.2 Asynchronous	Work that can complete later without requiring the current call to wait.
10.3 Callback	A function passed to another function to be called at a particular point.
10.4 Closure	A function with access to variables from its surrounding lexical scope.
10.5 DOM	The Document Object Model: the browser's structured representation of a page.
10.6 Event loop	Runtime coordination for queued callbacks and microtasks.

10.7 Hoisting	Processing of declarations before ordinary line-by-line execution; behavior varies by declaration type.
10.8 JSON	A text format commonly used to represent structured data.
10.9 Promise	An object representing the eventual success or failure of an asynchronous operation.
10.10 Prototype	An object used in JavaScript property lookup and inheritance.
10.11 Type coercion	Conversion of a value from one type to another, explicitly or implicitly.
10.12 WebSocket	An API/protocol for persistent, bidirectional client-server communication.
10.13 Worker	A separate execution context used to run certain work away from the browser's main thread.

11 · Appendix

Use this section to track completion and note source-specific details.

11.1 Progress checklist

11.1.1 Phase 1: syntax, operators, control flow, loops, functions, arrays, objects, errors, and DOM

11.1.2 Phase 2: Math, Date, date/time logic, JSON, and global functions

11.1.3 Phase 3: core behavior, modules, modern JavaScript, and advanced concepts

11.1.4 Phase 4: browser development, storage, APIs, accessibility, and async programming

11.1.5 Phase 5: runtime internals, interviews, testing, debugging, quality, deployment, Git/GitHub

11.1.6 Phase 6: complete three or four substantial projects

11.2 Editorial notes

11.2.1 The original phase order and topic coverage are retained; spelling and capitalization are normalized.

11.2.2 Array.to() appears in the source list; verify the intended meaning in the original learning context.

11.2.3 The source lists Basic DOM Manipulation before Basic Error Handling; both are included.

11.2.4 The source's “tickling” event-propagation term is interpreted as “capturing.”

11.2.5 Timeline, project checkpoints, prerequisites, and glossary are added planning aids.

Keep the roadmap flexible. Depth comes from building, debugging, explaining, and revisiting—not from checking every topic once.